

Employee Document Vault with Role-Based Access

FINAL PROJECT CONTRACT (v1.0)

This document defines the **single source of truth** for the project. All three team members must follow this contract exactly. Any implementation that deviates from this contract must be discussed and approved by all team members before changes are made.

The purpose of this contract is to eliminate integration issues while remaining faithful to the original project requirements.

1. Project Objective

Build a secure centralized document management system for employee records.

Supported document categories:

Offer Letters

Contracts

Payslips

Appraisal Reports

Compliance Certificates

The system functions as a private HR document vault where access is determined by role.

2. Core Functional Requirements

The system must support:

Upload Document
Download Document
List Documents
Soft Delete Document
Version History
Role-Based Access Control
Document Search
Document Sorting
Document Tagging
Audit Logging

3. AWS Region

All resources must be created in:

ap-south-2

No exceptions.

4. System Architecture Components

Storage Layer

Amazon S3

Amazon DynamoDB

Security Layer

Amazon Cognito User Pool

IAM Roles

JWT Authentication

Compute Layer

AWS Lambda

API Layer

Amazon API Gateway

Presentation Layer

React OR Plain HTML/JavaScript
Hosted on S3

5. User Roles

Exactly three roles exist.

Employee

Can access:

Own documents only

Manager

Can access:

Documents belonging to direct reports

HR_Admin

Can access:

All documents

6. Cognito Contract

User Groups

Exactly:

Employee

Manager

HR_Admin

Case-sensitive.

Required User Attributes

Every user must contain:

employee_id

role

Examples:

```
{
  "employee_id":"EMP001",
  "role":"Employee"
}
{
  "employee_id":"MGR001",
  "role":"Manager"
}
{
  "employee_id":"HR001",
```

```
"role":"HR_Admin"  
}
```

7. Employee and Manager ID Standards

Employee IDs

Format:

EMP001

EMP002

EMP003

Manager IDs

Format:

MGR001

MGR002

HR IDs

Format:

HR001

HR002

8. Manager Relationship Contract

DynamoDB Table

manager_mapping

Schema:

```
{  
  "manager_id":"MGR001",  
  "employee_id":"EMP001"  
}
```

Purpose:

Determines direct reports.

Managers may only access employees mapped to them.

9. Document Storage Structure

S3 Prefix Layout

Must always follow:

documents/{employee_id}/{document_type}/{filename}

Example:

documents/EMP001/payslip/jan2026.pdf

No alternative folder structures are allowed.

10. Allowed Document Types

Only these values:

offer_letter

contract

payslip

appraisal
compliance_certificate

Frontend, APIs, Lambda, and DynamoDB must all use these exact values.

11. S3 Configuration

Versioning

Must be enabled.

Lifecycle Rule

Applies to non-current versions.

Rule:

Older than 90 days
→ Move to S3 Standard-IA

12. Metadata Table Contract

DynamoDB Table

document_metadata

Partition Key

document_id

Required Attributes

```
{  
  "document_id":"UUID",  
  "employee_id":"EMP001",  
  "uploaded_by":"HR001",  
  "document_type":"payslip",  
  "s3_key":"documents/EMP001/payslip/jan2026.pdf",  
  "upload_timestamp":"2026-06-24T10:00:00Z",  
  "tags":["salary","2026"],  
  "is_deleted":false  
}
```

13. Document ID Standard

Generated by Upload Lambda.

Format:

UUID Version 4

Example:

550e8400-e29b-41d4-a716-446655440000

14. Audit Log Contract

DynamoDB Table

audit_log

Partition Key

audit_id

Required Attributes

```
{  
  "audit_id":"UUID",  
  "user_id":"EMP001",  
  "action":"DOWNLOAD",  
  "document_id":"UUID",  
  "timestamp":"2026-06-24T10:00:00Z"  
}
```

Allowed Actions

UPLOAD
LIST
DOWNLOAD
DELETE

Only these values.

15. Authorization Contract

All authorization occurs inside Lambda.

No frontend authorization decisions.

Employee Rules

Allowed:

`employee_id == requested employee_id`

Denied:

All other employees

Manager Rules

Allowed:

Employee exists in manager_mapping

Denied:

Employee not mapped

HR_Admin Rules

Allowed:

Everything

16. Upload Flow Contract

API

POST /upload

Request

```
{  
  "employee_id":"EMP001",  
  "document_type":"payslip",  
  "filename":"jan2026.pdf",  
}
```

```
"tags":["salary"]
}
```

Upload Lambda Responsibilities

1. Validate JWT
 2. Validate role permissions
 3. Generate document_id
 4. Build s3_key
 5. Create metadata record
 6. Generate pre-signed PUT URL
 7. Return URL
-

Response

```
{
  "document_id":"UUID",
  "upload_url":"..."
}
```

17. List Files Flow Contract

API

GET /files

Lambda Responsibilities

1. Validate JWT
2. Apply RBAC
3. Exclude deleted documents

4. Return metadata

Response

```
[
  {
    "document_id":"UUID",
    "document_type":"payslip",
    "upload_timestamp":"2026-06-24T10:00:00Z",
    "tags":["salary"]
  }
]
```

18. Download Flow Contract

API

GET /download/{doc_id}

Lambda Responsibilities

1. Validate JWT
 2. Verify ownership/access
 3. Fetch metadata
 4. Generate pre-signed GET URL
-

URL Expiry

15 Minutes

Response

```
{  
  "download_url": "..."  
}
```

19. Delete Flow Contract

API

DELETE /files/{doc_id}

Lambda Responsibilities

1. Validate JWT
2. Verify permissions
3. Update:

is_deleted = true

Must NOT:

Delete S3 object

Delete DynamoDB item

20. Version History Contract

Source:

S3 Versioning

No separate version table.

Version History Response

Must include:

Version ID

Last Modified Date

21. Search Contract

Search operates on metadata only.

Searchable fields:

document_type

tags

22. Sorting Contract

Supported sorts:

upload_timestamp

document_type

23. Timestamp Standard

All timestamps use:

ISO-8601 UTC

Example:

2026-06-24T10:00:00Z

24. Audit Logging Rules

Every successful operation must create an audit record.

Operations:

UPLOAD

LIST

DOWNLOAD

DELETE

25. Frontend Requirements

Must provide:

File Browser

Shows documents.

Folder Navigation

Grouped by document type.

Upload Button

Calls Upload API.

Download Button

Calls Download API.

Version History View

Displays S3 versions.

Search Bar

Filters metadata.

Sort Controls

Sort by:

Date

Type

Tags Display

Display tags stored in metadata.

26. Integration Test Data

Create users:

EMP001

EMP002

MGR001

HR001

Manager Mapping

MGR001 → EMP001

MGR001 → EMP002

Sample Documents

At least one:

Offer Letter

Contract

Payslip

Appraisal

Compliance Certificate

27. Mandatory Demo Scenarios

Scenario 1

Employee uploads document.

Expected:

Success

Scenario 2

Employee views own documents.

Expected:

Success

Scenario 3

Employee downloads own document.

Expected:

Success

Scenario 4

Employee accesses another employee's document.

Expected:

Access Denied

Scenario 5

Manager accesses direct report document.

Expected:

Success

Scenario 6

Manager accesses unrelated employee document.

Expected:

Access Denied

Scenario 7

HR_Admin accesses any document.

Expected:

Success

Scenario 8

Soft Delete document.

Expected:

is_deleted = true

Scenario 9

View version history.

Expected:

Multiple versions displayed

Scenario 10

Export audit logs.

Expected:

Minimum 10 audit records

Team Ownership

Team 1 — Security & Access

Owns:

Cognito
IAM Roles
manager_mapping
JWT Validation
Authorization Logic
API Gateway

Team 2 — Storage & Metadata

Owns:

S3
Versioning
Lifecycle Rules
document_metadata

Team 3 — Application Layer

Owns:

Upload Lambda
List Lambda
Download Lambda
Delete Lambda
audit_log
Frontend
Demo Preparation

This contract preserves the original project requirements while filling every mission-critical gap needed for successful parallel development and integration.

